Booting in Linux: Legacy BIOS versus UEFI

Modern Linux distributions prefer the UEFI booting process over legacy BIOS booting. Let's find out why.

Every time we press the power button on a computer, a complex chain of events begins—one that ends with our Linux desktop or server environment becoming fully functional. This process, called booting, involves multiple layers of interaction between firmware, storage devices, bootloaders, and the kernel.

For decades, computers relied on the BIOS (Basic Input/Output System) firmware, which operated in a simple but rigid way. However, as hardware evolved, BIOS showed its limitations: inability to handle disks larger than 2TB, restricted partitioning, and limited extensibility.

UEFI (Unified Extensible Firmware Interface) was introduced to overcome these challenges. It modernised the boot process with modular phases, support for larger disks, and features like Secure Boot.

Understanding the boot process

Before diving into BIOS and UEFI, it's important to outline the general flow of a boot sequence.

1. ***Power-on and firmware initialisation:*** The firmware (BIOS or UEFI) initialises hardware components and determines the boot device.
2. ***Bootloader loading:*** A small program (bootloader) is loaded from the disk into the memory of the computer.
3. ***Kernel loading:*** The bootloader loads the Linux kernel and initial RAM filesystem, which is called *initramfs*.
4. ***Kernel initialisation:*** The kernel initialises device drivers, mounts the root filesystem, and launches *systemd* or *init*, which brings up user space.

Legacy BIOS booting

Let's first explore the legacy BIOS booting process.

Power-on and CPU reset

When you press the power button, a reset signal is sent from the clock generator chip on the motherboard to the CPU reset pin. The signal is held high for a few clock cycles (depending on CPU type) and then goes low, which resets the CPU. On reset:

- CPU registers are initialised to a predetermined state.
- The instruction pointer (IP) is set to the reset vector address (a 20-bit address).
- This reset vector points to the start of the BIOS firmware.

At this stage, the CPU begins fetching and executing instructions directly from the BIOS chip (ROM/flash) without needing to copy them to RAM (execution-in-place).

BIOS execution and stages

BIOS is a collection of low-level programs stored in a ROM/flash chip on the motherboard. It has three internal roles.

- *Startup routines:* Initialising CPU, memory, chipset, and peripheral devices.
- *Service handling routines*: Software interrupts (INT calls) for OS and bootloader.
- *Hardware interrupt handling:* Basic interrupt handling for keyboard, timers, etc.

POST (Power-On Self-Test)

The first program BIOS runs is POST, which checks:

- CPU registers
- BIOS routines integrity
- Keyboard
- CMOS RAM
- Other essential hardware.

Once complete, BIOS copies itself from ROM to RAM (shadowing) for faster execution and then disables the